

Pebble: A Leaderless, Eventually Consistent Distributed Key-Value Store

Technical Whitepaper — Version 0.1

Abderrahmane Memmou

`neon-x-hub/pebble`

<https://github.com/neon-x-hub/pebble>

August 2026

Abstract

PEBBLE is an eventually consistent, leaderless, distributed key-value store implemented in TypeScript on Node.js. It replicates state across cluster nodes using a pull-based gossip protocol and resolves concurrent writes deterministically using a total-order Last-Write-Wins (LWW) relation. Cluster nodes and external clients are authenticated with Ed25519 digital signatures under a lightweight certificate-authority model. There is no leader election, no consensus algorithm, and no central coordinator: any node accepts reads and writes, and healthy nodes converge to identical state.

This whitepaper provides a complete formal description of PEBBLE's design: the mutation data model and algebra, the cryptographic layer, the LWW total-order proof, version-vector-based incremental synchronisation, the 4-step gossip protocol, node and client authentication handshakes, the TCP wire framing layer, persistence and crash recovery, a rigorous complexity analysis as a function of cluster size N , a catalogue of core system invariants, and a roadmap of known limitations and future work.

Contents

1	Introduction	4
1.1	Design Goals	4
1.2	Non-Goals (v0.1)	4
1.3	Contributions	5
1.4	Paper Organisation	5
2	System Architecture	5
2.1	Node Architecture	5
2.2	Package Dependency Graph	6
2.3	Data Flow Summary	7
3	Data Model and Mutation Algebra	7
3.1	The Mutation	7
3.2	Operations and Tombstones	8
3.3	The Mutation History	8
3.4	Current State	9
3.5	Mutation Wire Format	9
4	Cryptographic Layer	9
4.1	Node Identity and Key Derivation	9
4.2	Canonical JSON Serialisation	9
4.3	Mutation Signing and Verification	10
4.4	Moderator Credential System	10

5	Conflict Resolution: Deterministic Last-Write-Wins	11
5.1	The LWW Total Order	11
5.2	Proof: $\prec_{\text{lww}}$ is a Strict Total Order	11
5.3	Convergence Theorem	12
5.4	Implementation	12
5.5	Clock Skew Limitation	12
6	Version Vectors and Incremental Synchronisation	13
6.1	Definition	13
6.2	Vector Advancement	13
6.3	Missing-Range Computation	13
6.4	Monotonicity Invariant	14
6.5	Protocol Limits on Ranges	14
7	Gossip Protocol: 4-Step Symmetric Synchronisation	14
7.1	Session Overview	14
7.2	Step-by-Step Description	15
7.3	Protocol Properties	15
7.4	Gossip Target Selection	16
7.5	Health Monitoring	16
8	Node Authentication: 3-Way Handshake	16
8.1	Handshake Protocol	16
8.2	Step-by-Step Description	17
8.3	Security Properties	17
9	Client Authentication and Permission System	18
9.1	Authentication Protocol	18
9.2	Step-by-Step Description	18
9.3	Permission Enforcement	19
9.4	Security Properties	19
10	Wire Protocol: TCP Framing	19
10.1	Frame Format	19
10.2	Message Type Constants	20
10.3	Stream Decoder	20
10.4	Encoding	20
10.5	DoS Considerations	21
11	Persistence and Crash Recovery	21
11.1	On-Disk Layout	21
11.2	Append-Only Mutation Log	21
11.3	Snapshots	21
11.4	Crash Recovery Pipeline	22
11.5	Corrupted Log Tail	22
12	Complexity Analysis	23
12.1	Per-Node State Complexity	23
12.2	Version Vector and Missing-Range Computation	23
12.3	Gossip Round Complexity	23
12.4	Cluster-Wide Convergence Time	24
12.5	Gossip Session Load per Node	24
12.6	Handshake Complexity	25

12.7 Snapshot and Recovery Complexity	25
13 System Invariants	25
14 Known Limitations (v0.1)	26
14.1 Physical Wall-Clock Timestamps	26
14.2 Contiguous-Sequence Assumption in Version Vectors	26
14.3 Unbounded History Growth	27
14.4 No Multi-Key Atomicity	27
14.5 Static Cluster Membership	27
14.6 Gossip Fanout Fixed at 1	27
14.7 No Transport-Layer Encryption	27
15 Future Work	28
15.1 Hybrid Logical Clocks (HLC)	28
15.2 Sparse Version Vectors	28
15.3 History Compaction and Tombstone GC	28
15.4 Dynamic Cluster Membership	28
15.5 Multi-Key Transactions	29
15.6 TLS Transport Encryption	29
15.7 Configurable Gossip Fanout	29
15.8 Read Repair and Anti-Entropy	29
15.9 Read Consistency Modes	29
16 Conclusion	30

1 Introduction

Modern distributed applications routinely require storage systems that remain available during network partitions, tolerate node failures without service interruption, and scale horizontally without bottlenecks imposed by a central coordinator. The CAP theorem [2, 6] establishes that no distributed data store can simultaneously guarantee strong consistency, availability, and partition tolerance. Systems that choose availability and partition tolerance—the AP corner of the CAP triangle—must accept that nodes may temporarily diverge and must define a reconciliation strategy for concurrent writes.

Consensus-based systems such as Raft [13] and Paxos [11] achieve strong consistency by electing a leader that serialises all writes, at the cost of availability when a quorum cannot be reached, and of increased operational complexity. Leaderless, eventually consistent systems such as Amazon Dynamo [3] and Apache Cassandra sacrifice linearisability in favour of write availability on every node and partition tolerance, using vector clocks or timestamps to reconcile diverged state.

PEBBLE occupies this leaderless, eventually consistent space. It is a distributed key-value store built on a small set of composable primitives:

- **Immutable, signed mutations.** Every write or delete is captured as a cryptographically signed, globally unique record that is never modified after creation.
- **Deterministic Last-Write-Wins (LWW) conflict resolution.** A total order over mutations resolves concurrent writes without coordination, ensuring that all nodes independently compute the same winning value.
- **Pull-based gossip replication.** Nodes periodically exchange version summaries and transfer only the mutations each peer is missing, providing bandwidth-efficient eventual convergence.
- **Ed25519 authentication.** Both inter-node connections and external client sessions are authenticated with digital signatures, preventing rogue nodes or forged mutations from corrupting cluster state.

1.1 Design Goals

PEBBLE is designed to satisfy the following goals:

Leaderless. Any node accepts reads and writes. There is no leader election and no single point of failure.

Eventually consistent. All healthy nodes converge to identical state after the last write, given sufficient gossip rounds.

Authenticated. Mutations carry Ed25519 signatures; nodes and clients are verified before any state exchange occurs.

Simple. The core mechanisms (LWW, gossip, version vectors) are well understood and compose cleanly. The entire system is implemented in fewer than 3,000 lines of TypeScript.

Durable. Local mutations are written to an append-only log and synced to disk before acknowledgment. Nodes recover from crash by replaying the log from the last snapshot.

1.2 Non-Goals (v0.1)

The following properties are explicitly *not* provided in the current version and are discussed as future work in [section 15](#):

- Linearisability or sequential consistency.
- Multi-key atomic transactions.
- Dynamic cluster membership (node join/leave without restart).
- Transport-layer encryption (TLS).
- Bounded divergence guarantees under arbitrary clock skew.

1.3 Contributions

This whitepaper makes the following technical contributions:

1. A formal definition of the mutation algebra and the LWW total order, with a proof of convergence ([section 5](#)).
2. A precise specification of the version-vector-based incremental synchronisation scheme ([section 6](#)).
3. A complete description of the 4-step symmetric gossip protocol, with deadlock-freedom and idempotency arguments ([section 7](#)).
4. A complexity analysis of per-round bandwidth, state growth, and cluster-wide convergence time as a function of node count N ([section 12](#)).
5. A catalogue of eight formally stated system invariants ([section 13](#)).

1.4 Paper Organisation

[Section 2](#) describes the overall system architecture. [Section 3](#) defines the data model. [Section 4](#) covers the cryptographic layer. [Section 5](#) presents the LWW conflict resolution mechanism. [Section 6](#) formalises version vectors. [Section 7](#) specifies the gossip protocol. [Sections 8](#) and [9](#) describe node and client authentication. [Section 10](#) details TCP framing. [Section 11](#) covers persistence and crash recovery. [Section 12](#) provides complexity analysis. [Section 13](#) enumerates system invariants. [Sections 14](#) and [15](#) discuss limitations and future work. [Section 16](#) concludes.

2 System Architecture

PEBBLE is structured as a pnpm monorepo containing seven focused packages under the `@pebble` namespace. This section describes the high-level architecture, the two-port separation strategy, and the inter-package dependency graph.

2.1 Node Architecture

Each PEBBLE node runs two independent TCP services bound to separate ports:

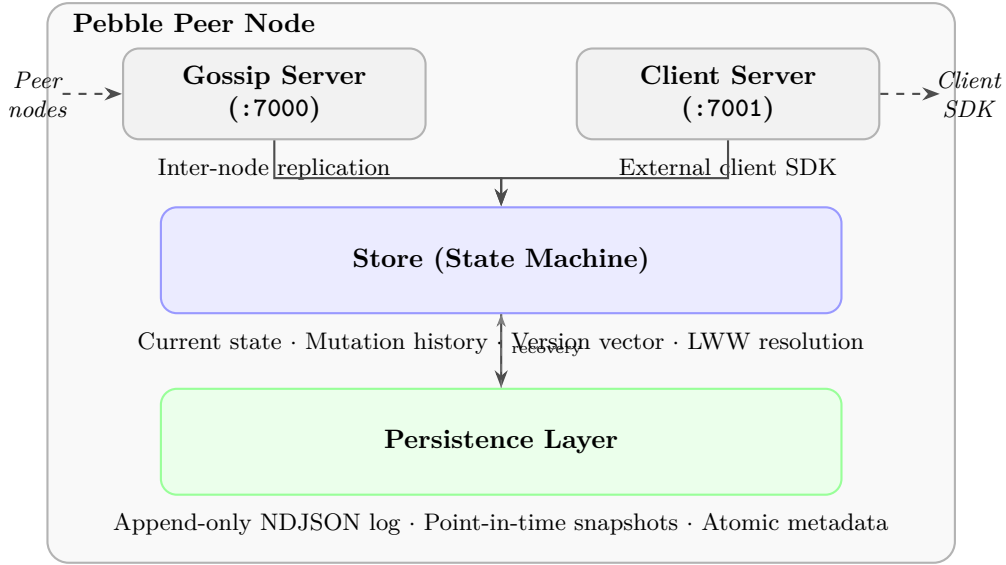


Figure 1: Internal architecture of a PEBBLE peer node.

Gossip Port (:7000). Dedicated exclusively to inter-node communication. All gossip sessions begin with a 3-way challenge-response handshake (section 8) to mutually authenticate the connecting peer. Once established, the connection carries PING/PONG health probes and the 4-step symmetric synchronisation protocol (section 7). External clients never access this port.

Client Port (:7001). Dedicated to the external `@pebble/client` SDK. Each connection undergoes a 2-message credential challenge (section 9) before any key-value operations are processed. Peer nodes never use this port.

This strict separation simplifies permission enforcement, enables independent scaling of client-facing and replication-facing capacity, and eliminates any ambiguity about which authentication path applies to a given connection.

2.2 Package Dependency Graph

PEBBLE is factored into seven packages that form a strict layered dependency graph with no cycles:

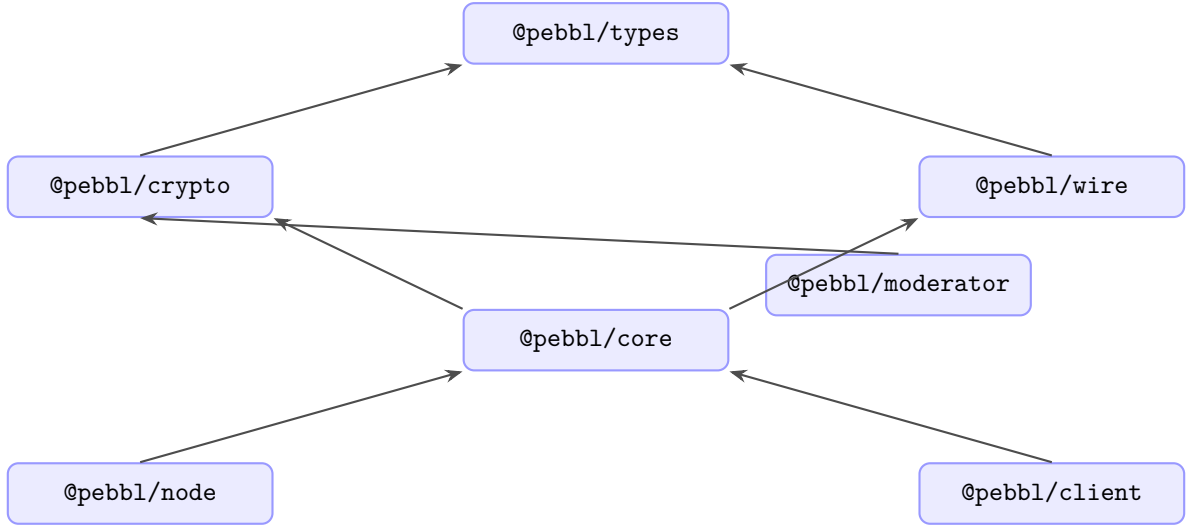


Figure 2: Inter-package dependency graph. Arrows point from dependent to dependency. `@pebble/moderator` is an offline utility with no runtime dependency on `@pebble/node`.

Table 1: Package responsibilities.

Package	Path	Responsibility
<code>@pebble/types</code>	<code>packages/types</code>	Shared domain interfaces, protocol constants, error classes. Zero
<code>@pebble/crypto</code>	<code>packages/crypto</code>	Ed25519 key generation, canonical JSON encoding, mutation an
<code>@pebble/wire</code>	<code>packages/wire</code>	Length-type-payload TCP framing and stateful stream decoder.
<code>@pebble/core</code>	<code>packages/core</code>	In-memory Store, LWW conflict resolution, append-only log, sn
<code>@pebble/node</code>	<code>packages/node</code>	Gossip engine, handshake, client server, health monitoring, top-
<code>@pebble/client</code>	<code>packages/client</code>	Client SDK with multiplexed request tracking and automatic au
<code>@pebble/moderator</code>	<code>packages/moderator</code>	Offline credential issuance and keypair generation.

2.3 Data Flow Summary

A write operation flows through the following components:

1. **Client SDK** sends a `PUT_REQUEST` over the client port.
2. **Client Server** verifies the client’s session, enforces `read-write` permission, and forwards the request to the **Store**.
3. **Store** creates an immutable signed mutation, applies LWW resolution, and emits a `mutation:applied` event.
4. **Pebble (orchestrator)** listens to the event and appends the mutation to the **Mutation Log** with `fsync`.
5. **Gossip Engine** periodically connects to healthy peers and transfers mutations they have not yet seen, as determined by comparing version vectors.

3 Data Model and Mutation Algebra

3.1 The Mutation

The fundamental unit of state change in PEBBLE is the *mutation*—an immutable, signed record of a single key-value operation.

Definition 3.1 (Mutation). A *mutation* is a 7-tuple

$$m = \langle \text{nodeId}, \text{seq}, \text{ts}, \text{op}, \text{key}, \text{val}, \sigma \rangle$$

where:

- $\text{nodeId} \in \mathcal{N}$ is the identifier of the node that created m .
- $\text{seq} \in \mathbb{Z}^+$ is a monotonically increasing counter local to nodeId : no two mutations from the same node share the same sequence number, and sequence numbers only increase across node restarts.
- $\text{ts} \in \mathbb{Z}^+$ is the wall-clock timestamp (milliseconds since the Unix epoch) at the time of creation.
- $\text{op} \in \{\text{put}, \text{delete}\}$ is the operation type.
- $\text{key} \in \mathcal{K}$ is the key being modified.
- $\text{val} \in \Sigma^* \cup \{\perp\}$ is the value: a non-empty UTF-8 string for **put**, and $\perp$ (null) for **delete**.
- $\sigma \in \{0, 1\}^{512}$ is the Ed25519 signature of the node over all other fields (see [section 4](#)).

Definition 3.2 (Mutation ID). The *mutation ID* of m is the pair $(\text{nodeId}, \text{seq})$, written $\text{id}(m) = \text{nodeId}:\text{seq}$. Mutation IDs are globally unique across the cluster:

$$\forall m, m' \in \mathcal{M} : \text{id}(m) = \text{id}(m') \implies m = m'.$$

This is enforced by the sequence collision check in the Store (see [section 13](#), Invariant [13.1](#)).

3.2 Operations and Tombstones

PEBBLE supports exactly two operations:

put(k, v) Associates key k with value $v \in \Sigma^*$.

delete(k) Creates a *tombstone* mutation for key k with $\text{val} = \perp$. The tombstone propagates through gossip like any other mutation, preventing deleted data from reappearing on nodes that have not yet received the deletion.

Definition 3.3 (Live key). A key k is *live* at node n at time t if the winning mutation for k in n 's store has operation **put**. A key is *deleted* if the winner is a tombstone. A key is *absent* if no mutation for k has been received.

Remark 3.4 (Tombstone replication). Tombstones must replicate through gossip exactly like writes. Without tombstone replication, a node that has received a **put** but not yet received the subsequent **delete** will continue to serve the stale value indefinitely. PEBBLE stores tombstones in the mutation history and transfers them during gossip sync. Tombstone GC is a v0.2 concern ([section 14](#)).

3.3 The Mutation History

Every PEBBLE node maintains an in-memory *mutation history*—a mapping from mutation IDs to the corresponding full mutation objects:

$$H : (\mathcal{N} \times \mathbb{Z}^+) \rightarrow \mathcal{M}$$

The history grows monotonically: once a mutation is added it is never removed (v0.1). The history enables exact range-based gossip: given a peer's version vector, the local node can enumerate precisely which mutations the peer is missing.

3.4 Current State

The *current state* is the projection of the history onto the winning mutation per key:

$$S : \mathcal{K} \rightarrow \mathcal{M}, \quad S(k) = \text{winner}(\{m \in H : m.\text{key} = k\})$$

where *winner* is defined by the LWW total order of [section 5](#). The current state is maintained incrementally: when a new mutation m for key k arrives, $S(k)$ is updated only if m wins over the existing $S(k)$.

3.5 Mutation Wire Format

Mutations are serialised as JSON objects when transmitted over the gossip protocol or stored in the mutation log. The canonical on-wire representation is:

```
1 {  
2   "nodeId":      "node-a",  
3   "sequence":    42,  
4   "timestamp":   17870600000000,  
5   "operation":   "put",  
6   "key":         "app:theme",  
7   "value":       "dark",  
8   "signature":   "3045022100..."  
9 }
```

Listing 1: Mutation JSON representation.

For `delete` mutations, `"value"` is `null`. The `signature` field covers all other fields via the canonical JSON encoding described in [section 4](#).

4 Cryptographic Layer

PEBBLE uses Ed25519 [1, 7] for all digital signatures. Ed25519 offers 128-bit security, fast verification, small 64-byte signatures, and no per-signature randomness requirement, making it well-suited for a high-throughput distributed system.

4.1 Node Identity and Key Derivation

Each node possesses a unique cryptographic identity generated once and persisted to `{dataDir}/identity.json`.

Definition 4.1 (Node identity). A node's identity is a triple $\mathcal{I} = (\text{nodeId}, pk, sk)$ where:

- (pk, sk) is an Ed25519 key pair. $pk \in \{0, 1\}^{256}$, $sk \in \{0, 1\}^{256}$ (the seed).
- $\text{nodeId} = \text{base64url}(\text{SHA-256}(pk))$ is the globally unique node identifier, derived deterministically from the public key.

The identity file is created on first start and loaded on subsequent starts, ensuring the same `nodeId` across restarts. The private key seed is stored in hex encoding and never transmitted over any network interface.

4.2 Canonical JSON Serialisation

To produce a deterministic byte sequence for signing, PEBBLE implements a canonical JSON encoder inspired by RFC 8785 [14].

Definition 4.2 (Canonical encoding). `canonicalEncode( $v$ )` serialises a JSON-compatible value v to a UTF-8 byte string such that:

1. Object keys are sorted lexicographically by their UTF-16 code units (matching the ECMA-Script `localeCompare` order).
2. No insignificant whitespace is emitted.
3. Numbers are represented in their shortest round-trip form.
4. String escaping follows JSON specification.

The canonical encoder is the only source of truth for the byte sequence presented to the signing primitive. Any serialisation discrepancy would cause valid signatures to appear invalid; therefore the encoder is tested with a deterministic golden-value test suite.

4.3 Mutation Signing and Verification

Definition 4.3 (Signable mutation fields). The *signable fields* of a mutation m are all fields except σ itself:

$$m_{\text{sign}} = \langle m.\text{nodeId}, m.\text{seq}, m.\text{ts}, m.\text{op}, m.\text{key}, m.\text{val} \rangle$$

Protocol 4.4 (Mutation signing). To create a signed mutation at node n with identity $\mathcal{I}_n$:

1. Assemble m_{sign} with all fields populated.
2. Compute $b = \text{canonicalEncode}(m_{\text{sign}})$.
3. Compute $\sigma = \text{Ed25519Sign}(sk_n, b)$.
4. Return $m = m_{\text{sign}} \cup \{\sigma\}$.

Protocol 4.5 (Mutation verification). To verify a mutation m received from a peer, given the originating node's public key pk_{nodeId} obtained from the cluster configuration:

1. Extract $m_{\text{sign}} = m \setminus \{\sigma\}$.
2. Compute $b = \text{canonicalEncode}(m_{\text{sign}})$.
3. Return $\text{Ed25519Verify}(pk_{\text{nodeId}}, b, m.\sigma)$.

If the originating `nodeId` is not present in the cluster configuration, verification fails with an `UnknownNodeError` before the signature is even checked.

Remark 4.6 (Startup recovery bypass). During crash recovery (section 11), mutations are replayed from the local on-disk log with signature verification *skipped*. This is safe because: (1) the log is written by the local node and protected by OS file permissions; (2) any mutation in the log was either created locally (and therefore valid) or received from a peer and verified at application time before being persisted. The `skipSignatureVerification` flag is never set for mutations received over the network.

4.4 Moderator Credential System

External client access is controlled through a lightweight certificate-authority (CA) model. The *moderator* is an offline entity that holds the cluster's root signing key.

Definition 4.7 (Client credential). A *client credential* is a 6-tuple signed by the moderator:

$$C = \langle \text{clientId}, pk_c, \text{perm}, t_{\text{issued}}, t_{\text{expires}}, \sigma_{\text{mod}} \rangle$$

where:

- *clientId* is a human-readable client identifier.
- $pk_c \in \{0, 1\}^{256}$ is the client's Ed25519 public key.
- $\text{perm} \in \{\text{read-only}, \text{read-write}\}$ is the access level granted.

- $t_{\text{issued}}, t_{\text{expires}} \in \mathbb{Z}$ are Unix millisecond timestamps ($t_{\text{expires}} = \perp$ for no expiry).
- $\sigma_{\text{mod}} = \text{Ed25519Sign}(sk_{\text{mod}}, \text{canonicalEncode}(\langle clientId, pk_c, perm, t_{\text{issued}}, t_{\text{expires}} \rangle))$.

Credential issuance is performed offline using `@pebble/moderator` and does not require modifying `cluster.json` or restarting running nodes. The moderator's public key pk_{mod} is stored in the cluster configuration and used by every node to verify incoming credentials.

Remark 4.8 (Security properties). The credential system provides:

- **Unforgeability:** only the holder of sk_{mod} can issue valid credentials.
- **Expiry enforcement:** nodes reject credentials where $\text{now}() > t_{\text{expires}}$.
- **Key binding:** during the AUTH handshake, the client proves possession of sk_c by signing a fresh nonce, preventing credential replay from a different identity.

5 Conflict Resolution: Deterministic Last-Write-Wins

When two mutations target the same key and arrive at a node in different orders on different peers, the nodes must converge to the same winner without coordination. PEBBLE achieves this through a deterministic total order on mutations—Last-Write-Wins (LWW).

5.1 The LWW Total Order

Definition 5.1 (LWW comparison). Define the relation $\prec_{\text{lww}}$ on mutations $a, b \in \mathcal{M}$ as follows:

$$a \prec_{\text{lww}} b \iff \begin{cases} a.ts < b.ts, \\ \text{or } a.ts = b.ts \wedge a.nodeId <_{\text{lex}} b.nodeId, \\ \text{or } a.ts = b.ts \wedge a.nodeId = b.nodeId \wedge a.seq < b.seq. \end{cases}$$

In prose: the mutation with the *higher timestamp* wins. Ties are broken by the *lexicographically later node ID*; this is arbitrary but stable across all nodes because every node applies the same comparison function. A further tiebreak on *higher sequence* handles the degenerate case of two mutations from the same node with the same timestamp (possible under fast local writes).

Definition 5.2 (Winner). Given two mutations a and b for the same key, the winner is:

$$\text{winner}(a, b) = \begin{cases} b & \text{if } a \prec_{\text{lww}} b, \\ a & \text{otherwise.} \end{cases}$$

5.2 Proof: $\prec_{\text{lww}}$ is a Strict Total Order

Theorem 5.3 ($\prec_{\text{lww}}$ is a strict total order on $\mathcal{M}$). *The relation $\prec_{\text{lww}}$ is irreflexive, asymmetric, transitive, and total on the set of all mutations $\mathcal{M}$.*

Proof. We verify each property:

Irreflexivity. $a \prec_{\text{lww}} a$ would require $a.ts < a.ts$ (false), or $a.ts = a.ts$ and $a.nodeId <_{\text{lex}} a.nodeId$ (false), or all equal and $a.seq < a.seq$ (false). Hence $a \not\prec_{\text{lww}} a$. ✓

Asymmetry (implies antisymmetry). Suppose $a \prec_{\text{lww}} b$. We show $b \not\prec_{\text{lww}} a$. If $a \prec_{\text{lww}} b$ holds because $a.ts < b.ts$, then $b.ts > a.ts$, so the first clause of $b \prec_{\text{lww}} a$ is false. The remaining clauses require $b.ts = a.ts$, which is also false. Hence $b \not\prec_{\text{lww}} a$. The remaining two cases proceed analogously by the asymmetry of $<$ and $<_{\text{lex}}$. ✓

Transitivity. Suppose $a \prec_{\text{LWW}} b$ and $b \prec_{\text{LWW}} c$. We show $a \prec_{\text{LWW}} c$ by case analysis on which clause governs each relation:

- If $a.ts < b.ts$ and $b.ts \leq c.ts$, then $a.ts < c.ts$, so the first clause of $a \prec_{\text{LWW}} c$ holds.
- If $a.ts = b.ts$ and $b.ts = c.ts$ (i.e., all equal), then the governing clause is on *nodeId* or *seq*, both of which are transitively ordered by the transitivity of $<_{\text{lex}}$ and $<$.
- Mixed cases (e.g., $a.ts < b.ts$ and $b.ts = c.ts$) collapse to $a.ts < c.ts$ by substitution. ✓

Totality. For any $a \neq b$, either $a.ts \neq b.ts$ (handled by the first clause), or $a.nodeId \neq b.nodeId$ (handled by the second, since $<_{\text{lex}}$ is total on strings), or $a.seq \neq b.seq$ (handled by the third, since $<$ is total on integers). The only remaining case is $a = b$, which is excluded. Hence exactly one of $a \prec_{\text{LWW}} b$ or $b \prec_{\text{LWW}} a$ holds for $a \neq b$. ✓ $\square$

5.3 Convergence Theorem

Theorem 5.4 (Convergence under LWW). *Let n_1, n_2 be two nodes that have received (in any order) the same set of mutations M for a key k . Both nodes independently compute the same current value for k .*

Proof. By [theorem 5.3](#), $\prec_{\text{LWW}}$ is a strict total order on $\mathcal{M}$. Therefore, the set $\{m \in M : m.key = k\}$ has a unique maximum element m^* under $\prec_{\text{LWW}}$. The function winner selects this maximum deterministically. Because winner is a function solely of the mutation set—not of arrival order—both nodes compute the same m^* regardless of the order in which they processed mutations from M . $\square$ $\square$

Corollary 5.5 (Eventual consistency). *If the gossip protocol ensures that every mutation created at any node is eventually delivered to every other node (i.e., the network is eventually connected), then all nodes eventually agree on the value of every key.*

5.4 Implementation

The comparison function is implemented in `packages/core/src/store/conflict.ts`:

```

1 export function compare(a: Mutation, b: Mutation): number {
2   if (a.timestamp !== b.timestamp) return a.timestamp - b.timestamp;
3   if (a.nodeId !== b.nodeId) return a.nodeId.localeCompare(b.nodeId);
4   return a.sequence - b.sequence;
5 }
6
7 export function winner(incumbent: Mutation, challenger: Mutation):
  Mutation {
8   return compare(challenger, incumbent) > 0 ? challenger : incumbent;
9 }

```

Listing 2: LWW comparison function (TypeScript).

A positive return value indicates the challenger wins; the Store updates `currentState[key]` only when the incoming mutation defeats the incumbent.

5.5 Clock Skew Limitation

The LWW order as defined relies on wall-clock timestamps. A node whose clock runs significantly behind can produce mutations that lose to older writes on other nodes, even if the losing mutation

was authored later in real time. PEBBLE v0.1 accepts this trade-off for simplicity. [Section 15](#) discusses Hybrid Logical Clocks (HLC) [\[10\]](#) as a path to causality-aware conflict resolution.

6 Version Vectors and Incremental Synchronisation

Version vectors [\[5, 12\]](#) are the mechanism by which PEBBLE nodes summarise their mutation history and compute precisely which mutations a peer is missing, without enumerating the entire history.

6.1 Definition

Definition 6.1 (Version vector). A *version vector* is a partial function

$$V : \mathcal{N} \rightarrow \mathbb{Z}^+$$

mapping each known node identifier to the *highest sequence number* seen from that node. Entries absent from V are treated as 0: $V[n] \triangleq 0$ if $n \notin \text{dom}(V)$.

Remark 6.2 (High-water mark semantics). PEBBLE v0.1 uses a *contiguous high-water mark*: $V[n] = s$ implies that this node has received *all* of node n 's mutations with sequence $1, 2, \dots, s$. This assumption is valid because:

1. Mutations from the same originating node are applied in the gossip response in sequence order.
2. A single TCP connection is used per gossip session; TCP guarantees in-order delivery within the session.

[section 14](#) discusses the implications of this assumption for out-of-order delivery.

6.2 Vector Advancement

When a mutation m is applied to the store, the local version vector is advanced:

Definition 6.3 (Vector advancement).

$$\text{advance}(V, m) : \quad V[m.\text{nodeId}] \leftarrow \max(V[m.\text{nodeId}], m.\text{seq})$$

The use of $\max$ ensures the vector is *monotonically non-decreasing*: once a sequence number is recorded for a node, it is never reduced. This satisfies Invariant [13.7](#) ([section 13](#)).

6.3 Missing-Range Computation

The core of incremental gossip synchronisation is computing which mutations one node needs from another.

Definition 6.4 (Missing ranges). Given the local node's vector V_A and the peer's vector V_B , the ranges that the peer is missing (i.e., mutations the local node has but the peer does not) are:

$$\text{missing}(V_A, V_B) = \{ (n, V_B[n] + 1, V_A[n]) \mid n \in \text{dom}(V_A) \wedge V_A[n] > V_B[n] \}$$

Each element $(n, \text{from}, \text{to})$ represents the sequence range $[\text{from}, \text{to}]$ for node n .

Example 6.5. Suppose $V_A = \{n_1 \mapsto 5, n_2 \mapsto 3\}$ and $V_B = \{n_1 \mapsto 3, n_3 \mapsto 2\}$.

Then:

$$\text{missing}(V_A, V_B) = \{(n_1, 4, 5), (n_2, 1, 3)\}$$

Node B is missing mutations 4 and 5 from n_1 , and all mutations 1–3 from n_2 . Note: n_3 entries in V_B that are absent from V_A are irrelevant to what A can send B .

Remark 6.6 (Symmetric computation). In a gossip round, both nodes compute missing ranges for the other:

- A computes $\text{missing}(V_B, V_A)$ — what A needs from B .
- B computes $\text{missing}(V_A, V_B)$ — what B needs from A .

After exchanging DIGESTs, each node has both vectors and can compute its own request set, enabling the symmetric 4-step protocol of [section 7](#).

6.4 Monotonicity Invariant

Invariant 6.7 (Monotonic version vectors). For any node n and any node identifier n' , $V_n[n']$ is non-decreasing over time. Formally, if event e_1 happens-before event e_2 at node n , then $V_n^{e_1}[n'] \leq V_n^{e_2}[n']$.

Proof. The advancement operation uses $\max$, which is monotonically non-decreasing. Vector entries are only ever written through advance, and no code path decreases a vector entry. $\square \square$

6.5 Protocol Limits on Ranges

To prevent a single gossip message from becoming arbitrarily large, PEBBLE caps the number of requested ranges per message at `limits.maxRangesPerRequest` (default: 100) and the number of mutations per MUTATIONS message at `limits.maxMutationsPerMessage` (default: 500). A node that is severely behind will catch up over multiple gossip rounds.

7 Gossip Protocol: 4-Step Symmetric Synchronisation

PEBBLE uses a *pull-based, symmetric gossip protocol* [\[4, 8\]](#) for replication. Every `gossipIntervalMs` milliseconds, each node selects one healthy peer (random selection among healthy nodes) and runs a complete sync session with it.

7.1 Session Overview

Each gossip session consists of exactly four steps, alternating message ownership between the initiator and responder. Both sides make progress in each round.

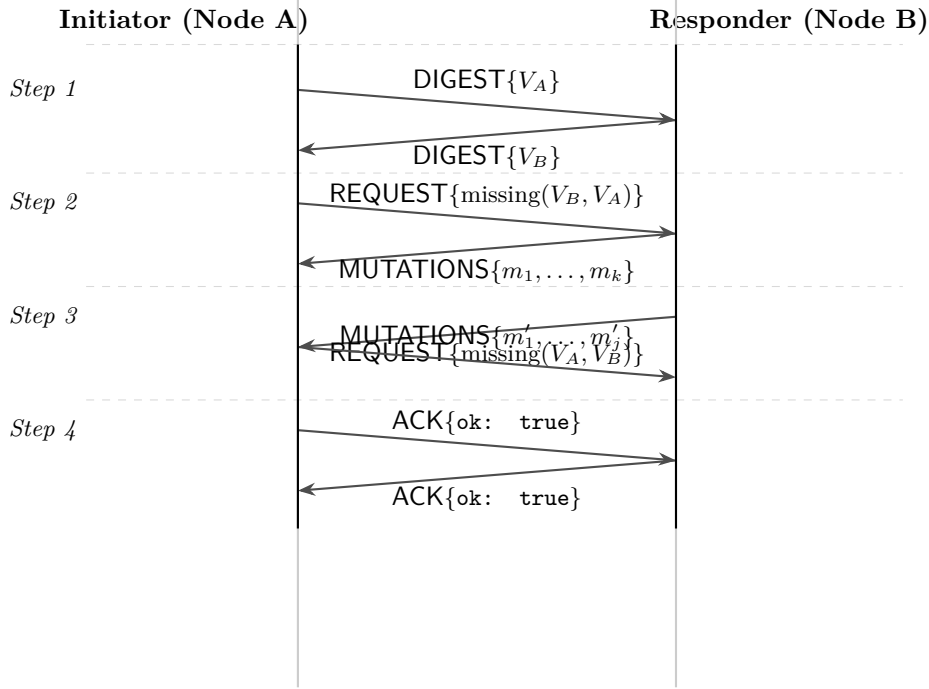


Figure 3: 4-step symmetric gossip protocol. After exchanging DIGESTs, each side requests and receives the mutations it is missing.

7.2 Step-by-Step Description

Step 1 — DIGEST exchange. Both nodes send their current version vectors simultaneously. After this step, each node holds both V_A and V_B and can independently compute the ranges that the other is missing.

Step 2 — Initiator requests and receives. The initiator computes $\text{missing}(V_B, V_A)$ (what A needs from B) and sends a `REQUEST` message. The responder looks up the requested mutations in its history and sends them in a `MUTATIONS` message. The initiator applies each received mutation through the store’s `applyMutation` pipeline (including signature verification).

Step 3 — Responder requests and receives. The responder computes $\text{missing}(V_A, V_B)$ (what B needs from A) and sends a `REQUEST`. The initiator replies with the matching mutations.

Step 4 — ACK. Both sides send `ACK` to signal successful completion. The gossip engine records a successful round and updates the peer’s health state.

7.3 Protocol Properties

Theorem 7.1 (Deadlock freedom). *The 4-step gossip protocol is deadlock-free.*

Proof. The message sequence is fixed and deterministic: initiator sends, then responder sends, alternating for exactly 4 sub-steps. Neither side waits for the other to initiate a send before completing its own step. Each `send/receive` call on a connection is backed by a buffered TCP socket; a send never blocks waiting for the remote side to call `receive` on the same message. Since the turn sequence is fixed and finite, no circular waiting condition can arise. $\square$ $\square$

Theorem 7.2 (Idempotency). *Applying a mutation that has already been applied is a no-op.*

Proof. Before applying any mutation m , `applyMutation` checks whether `id(m)` already exists in the history map H . If it does, and the stored content is byte-for-byte identical, the function returns `false` without modifying state. If the stored content differs (a sequence collision), an error is thrown (see Invariant 13.1). In the normal case, duplicate mutations are silently dropped, making gossip delivery idempotent. $\square$ $\square$

7.4 Gossip Target Selection

In each round, the `GossipEngine` calls `PeerHealth.selectGossipTarget()`, which returns a random healthy peer or `null` if none are available. This provides a simple, unbiased gossip fanout of 1 per interval. The expected number of rounds for a mutation to reach all N nodes is $O(N \log N)$ (see section 12).

7.5 Health Monitoring

The `Pinger` sends PING messages to all peers every `pingIntervalMs` milliseconds (default: 2,000 ms) on the gossip port. A peer is marked *unhealthy* if no successful communication has been observed in the past `unhealthyThresholdMs` milliseconds (default: 10,000 ms). Unhealthy peers are excluded from gossip target selection but remain in the configuration; they are automatically restored to healthy status upon the next successful handshake.

8 Node Authentication: 3-Way Handshake

Before any gossip data exchange, two PEBBLE nodes perform a mutual challenge-response handshake to authenticate each other and bind the connection to the correct cluster. This prevents rogue nodes from injecting mutations into a cluster and prevents cross-cluster contamination.

8.1 Handshake Protocol

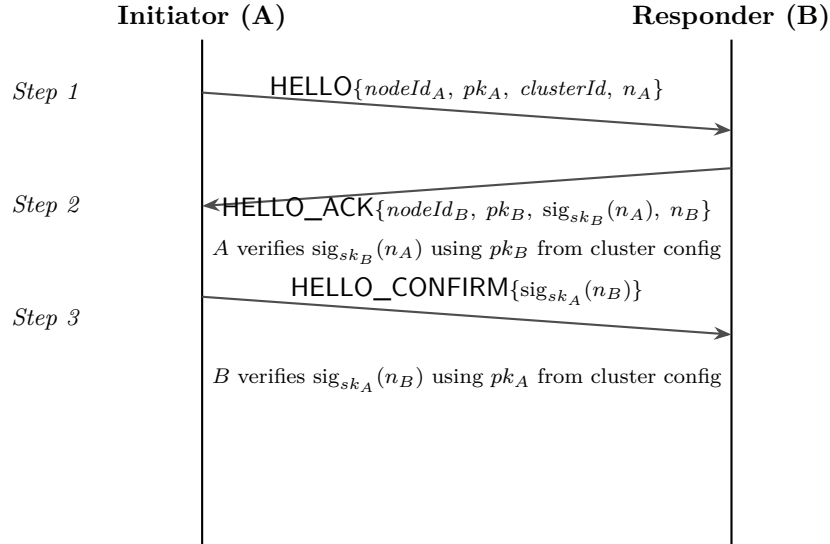


Figure 4: 3-way mutual authentication handshake between peer nodes. n_A, n_B are fresh 128-bit nonces; $sig_{sk}(x)$ denotes an Ed25519 signature of x under private key sk .

8.2 Step-by-Step Description

Step 1 — HELLO. The initiator sends its node identity and a fresh 128-bit random nonce n_A :

$$A \rightarrow B : \text{HELLO}\{nodeId_A, pk_A, clusterId, version, n_A\}$$

The cluster ID and protocol version are included so B can reject connections from different clusters or incompatible protocol versions immediately.

Step 2 — HELLO_ACK. The responder validates:

1. $clusterId$ matches its own configuration.
2. $version = 1$ (current protocol version).
3. $nodeId_A$ is present in the cluster configuration and pk_A matches.

If any check fails, B sends an ERROR and closes the connection. Otherwise, B generates its own nonce n_B , signs n_A with its private key, and replies:

$$B \rightarrow A : \text{HELLO_ACK}\{nodeId_B, pk_B, clusterId, sig_{sk_B}(n_A), n_B\}$$

A then verifies that $nodeId_B$ is in its cluster configuration, that pk_B matches, and that $sig_{sk_B}(n_A)$ is a valid Ed25519 signature.

Step 3 — HELLO_CONFIRM. The initiator signs n_B with its own private key and sends:

$$A \rightarrow B : \text{HELLO_CONFIRM}\{sig_{sk_A}(n_B)\}$$

B verifies the signature. Both sides have now mutually verified each other's identity. The connection proceeds to gossip sync.

8.3 Security Properties

Theorem 8.1 (Mutual authentication). *After a successful handshake, each node has verified that the peer holds the private key corresponding to the public key registered in the cluster configuration for that node's ID.*

Proof. B verifies $sig_{sk_A}(n_B)$ using pk_A from its local cluster configuration—not from the HELLO message itself. A successful verification proves A possesses sk_A such that $\text{Ed25519Verify}(pk_A, n_B, sig_{sk_A}(n_B)) = \top$. Symmetrically, A verifies B 's signature using the configuration-bound pk_B . $\square$ $\square$

Theorem 8.2 (Replay resistance). *A handshake transcript cannot be replayed by an adversary to establish a new authenticated connection.*

Proof. The nonces n_A and n_B are generated freshly for each connection attempt using a cryptographically secure random number generator. The signatures cover these nonces, so any replay of a captured transcript will fail verification because the nonces in the new challenge will differ. $\square$ $\square$

Theorem 8.3 (Cluster binding). *A node from cluster C_1 cannot successfully complete a handshake with a node configured for cluster $C_2 \neq C_1$.*

Proof. The cluster ID is checked in Step 2 before any signature is issued. A mismatch causes an immediate ERROR. $\square$ $\square$

9 Client Authentication and Permission System

External clients connect to the client port (:7001) and must present a moderator-issued credential before any key-value operations are processed. Authentication is performed through a 3-message challenge-response sequence.

9.1 Authentication Protocol

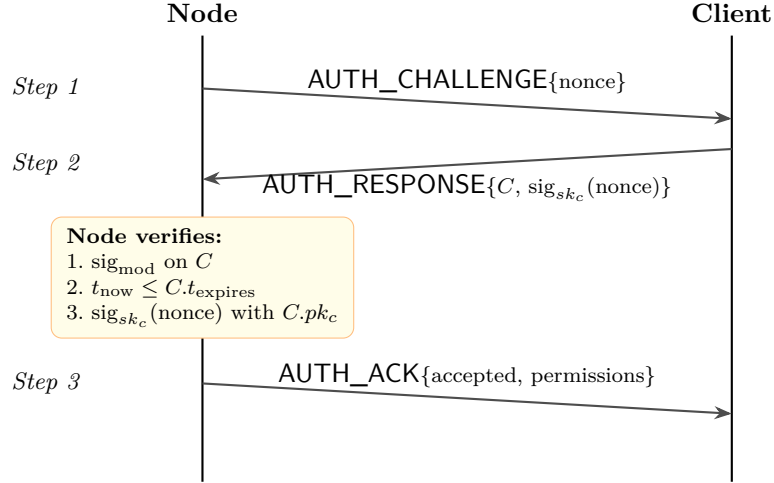


Figure 5: Client authentication protocol. C is the credential bundle; sk_c is the client's private key; sig_{mod} is the moderator's signature embedded in C .

9.2 Step-by-Step Description

Step 1 — AUTH_CHALLENGE. Upon connection, the node generates a fresh 128-bit nonce and sends it immediately:

Node $\rightarrow$ Client : `AUTH_CHALLENGE{nonce}`

The client has a configurable timeout to respond.

Step 2 — AUTH_RESPONSE. The client sends its credential bundle C (as defined in [definition 4.7](#)) and a signature of the nonce under its private key:

Client $\rightarrow$ Node : `AUTH_RESPONSE{C, sig_{sk_c}(nonce)}`

Step 3 — Verification and AUTH_ACK. The node performs three sequential checks:

1. **Moderator signature.** `Ed25519Verify(pk_{mod}, canonicalEncode(C \setminus \{\sigma_{mod}\}), C.\sigma_{mod})` must hold, using pk_{mod} from the cluster configuration.
2. **Credential expiry.** If $C.t_{expires} \neq \perp$, then `now()` $\leq C.t_{expires}$ must hold.
3. **Proof of key possession.** `Ed25519Verify(C.pk_c, nonce, sig_{sk_c}(nonce))` must hold.

If all checks pass, the node sends:

Node $\rightarrow$ Client : `AUTH_ACK{accepted :  $\top$ , C.perm}`

On any failure, the node sends an `ERROR` and closes the connection.

9.3 Permission Enforcement

After authentication, the node associates the connection with a session record $\{clientId, permissions, pk_c\}$. All subsequent requests are dispatched through the client handler, which enforces:

Table 2: Operation permission matrix.

Operation	read-only	read-write
GET_REQUEST	✓	✓
KEYS_REQUEST	✓	✓
ENTRIES_REQUEST	✓	✓
PUT_REQUEST	×	✓
DELETE_REQUEST	×	✓

A read-only client that attempts a PUT_REQUEST or DELETE_REQUEST receives a PERMISSION_DENIED error immediately; no state is modified.

9.4 Security Properties

The client authentication mechanism provides:

Unforgeability. Only the moderator can issue valid credentials. No network access or node coordination is required for issuance.

Key binding. The nonce signature proves the connecting client holds the private key matching the credential’s embedded public key, preventing credential transfer or replay by a third party.

Freshness. The nonce is generated per connection. Captured traffic cannot be replayed.

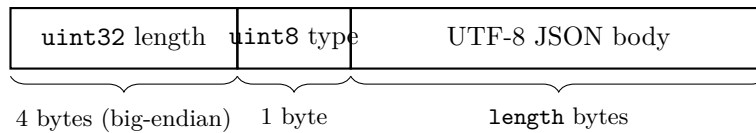
Expiry. Credentials can be issued with a finite lifetime; compromised credentials are automatically rejected after t_{expires} .

10 Wire Protocol: TCP Framing

Both the gossip protocol and the client protocol transmit JSON-encoded payloads over TCP using a shared length-type-payload framing layer implemented in `@pebble/wire`.

10.1 Frame Format

Definition 10.1 (Wire frame). A *wire frame* is a contiguous byte sequence with the following layout:



- **length** (uint32BE): byte length of the JSON body only. The 5-byte header is not included in this count.
- **type** (uint8): message type identifier (from `GossipMessageType` or `ClientMessageType`).
- **body**: compact UTF-8 JSON.

The total frame size is $5 + \text{length}$ bytes. The maximum permitted body size is `limits.maxMessageBytes` (default: 10 MiB).

10.2 Message Type Constants

Table 3: Gossip and client protocol message type bytes.

Gossip Protocol (port 7000)		Client Protocol (port 7001)	
Type	Byte	Type	Byte
HELLO	0x01	AUTH_CHALLENGE	0x10
HELLO_ACK	0x02	AUTH_RESPONSE	0x11
HELLO_CONFIRM	0x03	AUTH_ACK	0x12
PING	0x04	GET_REQUEST	0x20
PONG	0x05	PUT_REQUEST	0x21
DIGEST	0x06	DELETE_REQUEST	0x22
REQUEST	0x07	KEYS_REQUEST	0x23
MUTATIONS	0x08	ENTRIES_REQUEST	0x24
ACK	0x09	GET_RESPONSE	0x30
ERROR	0x0a	PUT_RESPONSE	0x31
		DELETE_RESPONSE	0x32
		KEYS_RESPONSE	0x33
		ENTRIES_RESPONSE	0x34
		ERROR	0x09

10.3 Stream Decoder

TCP is a byte-stream transport; a single data event may carry part of a frame, a complete frame, or multiple frames. The `FrameDecoder` class in `@pebble/wire` maintains a reassembly buffer and emits complete frames:

1. Incoming chunks are appended to an internal `Buffer`.
2. The decoder checks whether the buffer contains at least 5 bytes (the header).
3. If yes, it reads `length` from bytes 0–3 and `type` from byte 4.
4. If `length` bytes of body are available (i.e., buffer length $\geq 5 + \text{length}$), the frame is extracted and emitted.
5. If `length` exceeds `maxMessageBytes`, an error is emitted and the connection is closed, preventing OOM from a malformed or malicious message.
6. The consumed bytes are removed from the buffer; the process repeats for any remaining data.

This stateful push-based decoder correctly handles all TCP segmentation scenarios and is tested with single-byte push streams.

10.4 Encoding

Frame encoding is straightforward:

```
1 export function encodeFrame(type: number, payload: unknown): Buffer {
2   const body = Buffer.from(JSON.stringify(payload), "utf8");
3   const frame = Buffer.allocUnsafe(5 + body.byteLength);
4   frame.writeUInt32BE(body.byteLength, 0);
5   frame.writeUInt8(type, 4);
6   body.copy(frame, 5);
7   return frame;
8 }
```

Listing 3: Frame encoder (TypeScript).

10.5 DoS Considerations

The `maxMessageBytes` limit defends against memory exhaustion by a peer sending an oversized message. A frame with a `length` field exceeding this limit triggers an immediate connection close without attempting to buffer the body. This is the primary DoS surface in the current protocol; future versions may add per-connection rate limiting and total in-flight byte limits.

11 Persistence and Crash Recovery

Every PEBBLE node persists its state to an isolated data directory (`dataDir`). The persistence layer ensures that no acknowledged mutation is lost across node restarts and that the node can reconstruct its full in-memory state on startup.

11.1 On-Disk Layout

```
1 dataDir/  
2   identity.json           # Node keypair and nodeId (generated once)  
3   metadata.json          # Atomic sequence and vector counter  
4   mutations.log           # Append-only NDJSON mutation log (with fsync)  
5   snapshots/  
6     2026-08-20T00-00-00-000Z.snapshot.json  
7     2026-08-21T00-00-00-000Z.snapshot.json  
8     ...
```

Listing 4: Data directory layout.

11.2 Append-Only Mutation Log

The `MutationLog` (`packages/core/src/persistence/log.ts`) appends each accepted local mutation to `mutations.log` as a single newline-delimited JSON record (NDJSON format). Each append is followed by an `fsync` call to flush the kernel buffer to durable storage before the mutation is acknowledged to the caller.

Definition 11.1 (Durability guarantee). A mutation m is *durable* at node n if m has been serialised to `mutations.log` and the `fsync` call has returned. Durable mutations survive process crashes, OS crashes, and power failures (assuming the storage medium respects `fsync` semantics).

Remark 11.2 (Gossip-received mutations). Mutations received via gossip from peer nodes are applied to the in-memory store and emitted as `mutation:applied` events. The orchestrator (`Pebble` class) listens for these events and appends received mutations to the log as well, ensuring that any mutation in the in-memory state is also represented on disk.

11.3 Snapshots

Taking a snapshot serialises the entire current state (including tombstones) and the current version vector to a JSON file in `snapshots/`:

Definition 11.3 (Snapshot). A *snapshot* $\mathcal{S}$ is a tuple

$$\mathcal{S} = \langle \text{version}, t_{\mathcal{S}}, V, \text{nextSeq}, S \rangle$$

where V is the version vector at snapshot time, nextSeq is the next local sequence number, and S is the current state map ($\text{key} \rightarrow \text{winning mutation}$, including tombstones).

Snapshot filenames are ISO 8601 timestamps safe for all operating systems (colons replaced with hyphens). Lexicographic sort of filenames corresponds to chronological order, enabling simple latest-snapshot selection.

Snapshots serve two purposes:

1. **Faster recovery:** recovery needs only to replay log entries after the snapshot timestamp, rather than the full history.
2. **Log truncation (future):** once a snapshot is taken, older log entries that predate the snapshot can safely be discarded ([section 15](#)).

11.4 Crash Recovery Pipeline

On node startup, the `recover` function reconstructs in-memory state from the on-disk data:

Algorithm 1 Node crash recovery (`recover`)

Require: Node `dataDir`, empty `store`, `nodeId`

```

1:  $\mathcal{S} \leftarrow \text{loadLatestSnapshot}(\text{dataDir})$ 
2:  $\mathcal{M} \leftarrow \text{loadMetadata}(\text{dataDir})$ 
3:  $\text{nextSeq} \leftarrow \mathcal{M}.\text{nextLocalSequence}$ 
4: if  $\mathcal{S} \neq \perp$  then
5:   store.importSnapshot( $\mathcal{S}.S$ ,  $\mathcal{S}.V$ ,  $\mathcal{S}.\text{nextSeq}$ )
6:    $\text{nextSeq} \leftarrow \max(\text{nextSeq}, \mathcal{S}.\text{nextSeq})$ 
7: else
8:   store.setNextLocalSequence( $\text{nextSeq}$ )
9: end if
10:  $\text{log} \leftarrow \text{MutationLog}(\text{dataDir}).\text{readAll}()$ 
11: for each mutation  $m$  in  $\text{log}$  do
12:   store.applyMutation( $m$ ,  $\{\text{skipSignatureVerification} : \top\}$ )
13:   if  $m.\text{nodeId} = \text{nodeId}$  then
14:      $\text{nextSeq} \leftarrow \max(\text{nextSeq}, m.\text{seq} + 1)$ 
15:   end if
16: end for
17: store.setNextLocalSequence( $\text{nextSeq}$ )

```

Correctness argument. The log replay is idempotent: duplicate mutations (e.g., those already covered by the snapshot) are silently skipped by the store’s duplicate detection. Mutations that arrived after the snapshot timestamp but before the crash are replayed in full. The *nextSeq* watermark is advanced to be strictly greater than any local sequence seen, preventing sequence number reuse across restarts (Invariant 13.8).

11.5 Corrupted Log Tail

If the node crashed mid-write, the last line of `mutations.log` may be a truncated, invalid JSON record. The log reader handles this by discarding any trailing partial line (lines that do not parse as valid mutations are skipped with a warning). Since the line was not yet synced (the process crashed before `fsync` returned), it was never acknowledged, and discarding it preserves the durability guarantee.

12 Complexity Analysis

This section analyses the computational and communication complexity of PEBBLE as a function of the cluster size N (number of nodes), the total number of distinct keys K , and the total number of mutations M in the cluster history.

12.1 Per-Node State Complexity

Table 4: In-memory state sizes per node.

Structure	Size	Notes
Current state map (S)	$O(K)$	One winning mutation per key, including tombstones
Mutation history (H)	$O(M)$	All mutations ever received; no GC in v0.1
Version vector (V)	$O(N)$	One entry per known node

The total in-memory footprint per node is $O(K + M + N)$. In practice $M \gg K$ after steady-state operation (many mutations per key), so the dominant term is $O(M)$.

Remark 12.1 (History growth). Because PEBBLE v0.1 retains the full mutation history without compaction, memory usage grows without bound as long as mutations are generated. This is the primary scalability concern in the current version; see [section 14](#) and [section 15](#) for the planned GC mechanism.

12.2 Version Vector and Missing-Range Computation

Proposition 12.2. *Computing $\text{missing}(V_A, V_B)$ requires $O(N)$ time and produces at most N range entries.*

Proof. The computation iterates over $\text{dom}(V_A)$, which has at most N entries (one per node). For each entry, a constant-time comparison and conditional append are performed. The result has at most N elements (one per originating node). $\square$ $\square$

12.3 Gossip Round Complexity

Table 5: Per-gossip-round message complexity.

Phase	Messages	Max Payload
DIGEST exchange	2 messages	$O(N)$ bytes each
REQUEST (initiator)	1 message	$O(N)$ ranges
MUTATIONS (from B)	1 message	$O(M_{\max})$ mutations
REQUEST (responder)	1 message	$O(N)$ ranges
MUTATIONS (from A)	1 message	$O(M_{\max})$ mutations
ACK $\times 2$	2 messages	$O(1)$ each
Total	8 messages	$O(N + M_{\max})$ bytes

where $M_{\max} = \text{limits.maxMutationsPerMessage}$ (default: 500).

Worst-case bandwidth. If a node is starting fresh with no history and connects to a node with M total mutations, it will require $\lceil M/M_{\max} \rceil$ gossip rounds to catch up, each transferring $O(M_{\max})$ mutations. Total catch-up bandwidth: $O(M)$.

Incremental (steady-state) bandwidth. During steady-state operation with ΔM new mutations since the last sync with a given peer, a single gossip round transfers $O(\Delta M)$ mutations. This is the common case.

12.4 Cluster-Wide Convergence Time

PEBBLE uses random gossip fanout of 1 per node per interval. This is equivalent to the *push-pull* gossip model studied by Demers et al. [4].

Theorem 12.3 (Expected convergence time). *Starting from a single node holding a mutation, the expected number of gossip rounds for the mutation to reach all N nodes is $\Theta(N \log N)$, assuming each node selects a uniformly random healthy peer in each round.*

Sketch. This follows from the standard coupon-collector analysis of epidemic broadcasting. In round r , let k be the number of informed nodes. Each of the $N - k$ uninformed nodes is contacted by an informed node with probability k/N per round. By linearity of expectation and the harmonic series, the expected total rounds until all nodes are informed is:

$$\mathbb{E}[\text{rounds}] = N \sum_{k=1}^N \frac{1}{k} = \Theta(N \log N).$$

□

□

Table 6: Expected convergence rounds for representative cluster sizes with `gossipIntervalMs` = 1,000 ms.

N	$\mathbb{E}[\text{rounds}]$	Approx. wall time		Note
3	≈ 5.5	≈ 5.5 s		Small dev cluster
5	≈ 11	≈ 11 s		Small production
10	≈ 29	≈ 29 s		Medium cluster
20	≈ 72	≈ 72 s		Large cluster
50	≈ 225	≈ 225 s		Use higher fanout
100	≈ 519	≈ 519 s		Use higher fanout

Remark 12.4 (Fanout scaling). For clusters larger than ~ 20 nodes, a gossip fanout greater than 1 (selecting $f > 1$ peers per round) reduces convergence time to $\Theta(\log N / \log f)$ rounds. This is a planned improvement (section 15).

12.5 Gossip Session Load per Node

Each node initiates exactly one outbound gossip session per `gossipIntervalMs` interval and can receive an unbounded number of inbound sessions (one per active peer). In a cluster of N nodes:

- **Outbound load:** $O(1)$ session per node per interval (constant).
- **Inbound load:** $O(N)$ sessions per interval, since every other node may choose this node as its target.

Each inbound session occupies a dedicated TCP connection for the duration of the 4-step protocol. Because gossip sessions are short-lived and sequential per initiator, the server must handle up to $N - 1$ concurrent inbound connections. The Node.js event loop can sustain hundreds of concurrent lightweight connections without saturation.

12.6 Handshake Complexity

The 3-way handshake (section 8) requires 3 round trips and 4 Ed25519 operations (2 signs, 2 verifies) per connection. All operations are $O(1)$ in node count and dominate only the connection-establishment phase.

12.7 Snapshot and Recovery Complexity

Table 7: Persistence operation complexity.

Operation	Complexity
Snapshot write	$O(K)$ — serialise current state (one entry per key)
Snapshot read	$O(K)$ — deserialise and validate
Log append	$O(1)$ amortised — single NDJSON line + fsync
Log replay	$O(M)$ — replay all entries (bounded by M)
Recovery (total)	$O(K + M)$ — snapshot load + log replay

13 System Invariants

PEBBLE maintains eight core invariants across all subsystems. These invariants collectively guarantee correctness, security, and eventual consistency.

Invariant 13.1 (Global Uniqueness). The pair $(nodeId, seq)$ uniquely identifies at most one mutation across the entire cluster:

$$\forall m, m' \in \mathcal{M} : m.nodeId = m'.nodeId \wedge m.seq = m'.seq \implies m = m'.$$

Enforcement: The Store’s `applyMutation` checks whether `id(m)` is already in the history map H . If it is and the content differs, a `SequenceCollisionError` is thrown and the mutation is rejected. Only exact duplicates (same content) are silently accepted as idempotent.

Invariant 13.2 (Immutability). Once a mutation m is stored in H , its fields are never modified.

$$\forall t, t' > 0 : m \in H^t \implies H^{t'}[id(m)] = m.$$

Enforcement: All mutation fields are declared `readonly` in TypeScript. The in-memory map stores references to frozen objects; no code path modifies a mutation after creation.

Invariant 13.3 (Authenticity). Every mutation stored in H that originated from a remote peer carries a valid Ed25519 signature from the node identified by $m.nodeId$:

$$\forall m \in H_{\text{remote}} : \text{Ed25519Verify}(pk_{m.nodeId}, \text{canonicalEncode}(m_{\text{sign}}), m.\sigma) = \top.$$

Enforcement: `applyMutation` with `skipSignatureVerification = false` (the default for all network paths) performs verification before committing. An invalid signature raises `InvalidSignatureError`.

Invariant 13.4 (Idempotence). Applying the same mutation m to a store multiple times produces the same result as applying it once:

$$\forall m : \text{apply}(\text{apply}(S, m), m) = \text{apply}(S, m).$$

Enforcement: If `id(m) ∈ H`, `applyMutation` returns immediately without touching state. Gossip may deliver the same mutation multiple times (e.g., via two independent peers); idempotence ensures no harm.

Invariant 13.5 (Deterministic Ordering). Given the same set of mutations M for a key k , every node independently computes the same winning mutation:

$$\forall n_1, n_2 : H_{n_1} \cap \{m : m.key = k\} = H_{n_2} \cap \{m : m.key = k\} \implies S_{n_1}(k) = S_{n_2}(k).$$

Enforcement: The LWW `compare` function is a pure function of the mutation’s fields. It uses no node-local state, no random values, and no external communication. By [theorem 5.4](#), this implies convergence.

Invariant 13.6 (Tombstone Replication). If a `delete` mutation m_d for key k exists in the cluster, it is eventually present in every node’s history H and wins over any prior `put` mutation:

$$\exists n : m_d \in H_n \implies \forall n' : \text{eventually } m_d \in H_{n'}.$$

Enforcement: Tombstones are stored in H and replicated through gossip exactly like put mutations. The LWW order ensures a tombstone with a higher timestamp wins over prior puts (absent concurrent puts with a higher timestamp).

Invariant 13.7 (Monotonic Version Vectors). Version vector entries are non-decreasing over time:

$$\forall n, n' : V_n^t[n'] \leq V_n^{t+\epsilon}[n'].$$

Enforcement: The `advanceVector` function uses $V[n] \leftarrow \max(V[n], m.seq)$. No code path decreases a vector entry. Proved in [Invariant 6.7](#).

Invariant 13.8 (Monotonic Sequences). The local sequence counter at a node never repeats or regresses across restarts:

$$\forall t, t' > 0, t < t' : nextSeq_n^t \leq nextSeq_n^{t'}.$$

Enforcement: The crash recovery algorithm ([algorithm 1](#)) sets $nextSeq$ to the maximum of the metadata value, the snapshot value, and one more than the highest local sequence seen in the log. A node starting with an empty log and metadata initialises $nextSeq = 1$.

14 Known Limitations (v0.1)

The current version of PEBBLE makes several deliberate simplifications that limit its applicability in certain scenarios. This section catalogues each limitation honestly, explains why it exists, and references the corresponding future-work item.

14.1 Physical Wall-Clock Timestamps

Impact: The LWW total order uses $m.ts$ as the primary comparison key ([definition 5.1](#)). Nodes with skewed clocks can produce mutations that lose to older writes from well-clocked nodes, even when the skewed node’s write is causally later.

Example: Node A ’s clock is 5s behind. Node B writes $k = v_1$ at $t = 1000$. Node A writes $k = v_2$ at $t = 997$ (real time 1002). Despite A ’s write being causally later, v_1 wins because $1000 > 997$.

Severity: Low in practice for clusters with NTP-synchronised clocks (typical drift < 100 ms), but potentially problematic in WAN deployments or environments with unreliable time sources.

Planned fix: Hybrid Logical Clocks [\[10\]](#) ([section 15](#)).

14.2 Contiguous-Sequence Assumption in Version Vectors

Impact: PEBBLE’s version vector records only the highest sequence seen from each node, implying that all lower sequences have been received. This assumption holds only when mutations arrive in order within a single TCP connection. If future versions add parallel or unreliable delivery, the high-water mark can skip over missing mutations.

Severity: Currently safe, because all gossip delivery occurs over a single ordered TCP stream per session. Not safe if multi-path delivery or UDP-based gossip is introduced without a corresponding change to the vector representation.

Planned fix: Sparse version vectors using bitmap or interval-tree representation ([section 15](#)).

14.3 Unbounded History Growth

Impact: The mutation history H retains every mutation ever seen. Memory usage grows linearly with total mutation count M with no bound. On-disk log size also grows unboundedly.

Severity: High for long-running deployments with high write throughput. A node writing 1,000 mutations/second accumulates ~ 86 million mutations per day, each consuming roughly 200 bytes, resulting in ~ 17 GB/day.

Planned fix: Snapshot-based GC that discards log entries and history entries for mutations older than the latest snapshot, retaining only the winning mutation per key ([section 15](#)).

14.4 No Multi-Key Atomicity

Impact: PEBBLE does not support multi-key transactions. A client performing a sequence of puts over a key group has no guarantee that an intermediate state (some keys updated, others not) is invisible to concurrent readers.

Severity: Applications requiring atomic updates to multiple keys (e.g., transferring a value from key A to key B) cannot be implemented correctly with PEBBLE v0.1.

Planned fix: Optimistic concurrency control or saga patterns ([section 15](#)).

14.5 Static Cluster Membership

Impact: The set of nodes is defined at startup via `cluster.json`. Adding or removing a node requires updating the configuration file and restarting all nodes to reload it. There is no online join/leave mechanism.

Severity: Moderate. Affects operational flexibility but not correctness; the cluster continues to function with the existing node set.

Planned fix: A CRDT-based membership view with Paxos-free join/leave propagation ([section 15](#)).

14.6 Gossip Fanout Fixed at 1

Impact: Each node initiates exactly one gossip session per interval. Convergence time grows as $\Theta(N \log N)$ ([theorem 12.3](#)), which becomes impractically slow for large clusters ($N > 50$).

Severity: Low for small clusters; significant for large ones.

Planned fix: Configurable fanout $f > 1$, reducing convergence to $\Theta(\log N / \log f)$ rounds.

14.7 No Transport-Layer Encryption

Impact: Gossip and client traffic is transmitted as plaintext JSON over raw TCP. An on-path adversary can read mutation contents and client credentials in transit.

Severity: High in untrusted network environments. The Ed25519 authentication layer prevents *impersonation* and *mutation injection*, but not *eavesdropping*.

Planned fix: TLS for both the gossip and client ports ([section 15](#)).

15 Future Work

This section outlines the planned evolution of PEBBLE beyond v0.1, grouped by impact and estimated implementation complexity.

15.1 Hybrid Logical Clocks (HLC)

Motivation: Replace physical wall-clock timestamps with Hybrid Logical Clocks [10] to capture causality between events, even under clock skew.

Approach: Each node maintains an HLC state (l, c) where l is the maximum of the local physical clock and the highest l seen in received messages, and c is a counter that breaks ties at the same l value. The HLC advances monotonically and satisfies $l \geq \text{now}()$ at all times, ensuring that causally later events always receive larger HLC values.

Impact: The LWW comparison function (definition 5.1) would replace $m.ts$ with the HLC value, eliminating clock-skew-induced inversions. The mutation wire format would add an HLC component alongside the physical timestamp. Backward compatibility requires a protocol version bump.

15.2 Sparse Version Vectors

Motivation: Remove the contiguous-sequence assumption to support out-of-order mutation delivery and multi-path replication.

Approach: Replace the high-water mark integer $V[n]$ with a compact set representation of received sequence numbers (e.g., a sorted interval list or a roaring bitmap [9]). The `getMissingRanges` function would compute gaps in the received set rather than the single suffix above the watermark.

Impact: Memory overhead per node increases by a constant factor. Gossip requests become slightly larger (a list of gaps rather than a single range). Enables future UDP-based or multi-path gossip without correctness risk.

15.3 History Compaction and Tombstone GC

Motivation: Bound memory and on-disk storage by discarding history entries for mutations that are no longer needed for gossip sync.

Approach: Periodically take a snapshot and promote it to a *stable checkpoint*. All history entries for mutations covered by the checkpoint can be discarded. Gossip peers that have not yet received those mutations must be directed to fetch the snapshot instead of individual mutations (a form of state transfer).

Tombstone GC requires additional coordination: a tombstone can only be discarded if all nodes in the cluster have seen it. A simple mechanism is a cluster-wide minimum version vector: $V_{\min}[n] = \min_i V_i[n]$ over all nodes. History entries with sequence $\leq V_{\min}[n]$ are safe to discard.

Impact: Bounds $|H|$ to $O(K)$ after compaction (one entry per key). Requires nodes that fall far behind to perform a full state transfer rather than incremental gossip.

15.4 Dynamic Cluster Membership

Motivation: Allow nodes to join or leave the cluster without restarting all nodes or manually editing `cluster.json`.

Approach: Represent the cluster membership as a CRDT (e.g., a 2P-Set or OR-Set) replicated

through the gossip protocol. A joining node broadcasts a signed join request; existing nodes add it to the membership CRDT and begin gossiping with it. A leaving node broadcasts a signed leave message. The moderator signs membership changes to prevent rogue nodes from injecting themselves.

Impact: Eliminates the need for out-of-band cluster reconfiguration. Requires all nodes to run a compatible membership CRDT implementation.

15.5 Multi-Key Transactions

Motivation: Provide atomic read-modify-write and multi-key update semantics for applications that require them.

Approach: Two candidate approaches:

1. **Optimistic concurrency control (OCC):** The client reads a set of keys with their current version vectors, applies changes, and submits a conditional write that is rejected if any observed version has changed. Conflicts are retried by the client.
2. **Saga pattern:** Multi-key updates are composed as a sequence of compensating single-key mutations. Partial failures leave a detectable intermediate state that can be rolled back.

Full distributed serialisable transactions would require a consensus layer (contradicting the leaderless design goal) and are not planned.

15.6 TLS Transport Encryption

Motivation: Protect mutation contents and client credentials from on-path eavesdroppers.

Approach: Wrap both the gossip port and the client port in TLS 1.3. Peer authentication during the TLS handshake can reuse the existing Ed25519 keys, eliminating the need for a separate PKI (mTLS with self-signed certs derived from the Ed25519 identity). The application-level handshake ([section 8](#)) could be simplified or merged with the TLS handshake.

15.7 Configurable Gossip Fanout

Motivation: Reduce convergence time for large clusters by initiating $f > 1$ gossip sessions per interval.

Approach: Expose a `gossipFanout` parameter in `cluster.json`. The `GossipEngine` selects f random healthy peers per interval and runs sessions concurrently. Expected convergence time drops to $\Theta(\log_f N)$ rounds, as each round covers an f -fold more ground.

15.8 Read Repair and Anti-Entropy

Motivation: Detect and repair state divergence that persists beyond the normal gossip window (e.g., due to prolonged partitions or log corruption on a specific node).

Approach: A periodic anti-entropy pass computes a Merkle tree over the current state and compares digests with all peers. Divergent subtrees are repaired by re-exchanging the affected mutations. This provides a background consistency guarantee independent of the gossip fanout.

15.9 Read Consistency Modes

Motivation: Offer stronger read guarantees for applications that need them, without abandoning the leaderless design for writes.

Approach: Two optional read modes:

Read-your-writes: After a write at node n , route subsequent reads for the same key to n until the version has propagated.

Monotonic reads: Track a client-side version vector; reject reads from nodes whose vector does not dominate the client’s last-observed vector.

Both modes are implementable without consensus and add only client-side bookkeeping overhead.

16 Conclusion

This whitepaper has presented PEBBLE—a leaderless, eventually consistent, distributed key-value store built on a small set of composable primitives.

The system’s correctness rests on three interlocking formal foundations. First, the LWW total order (definition 5.1, theorem 5.3) ensures that any set of mutations for a key has a unique maximum element, and that every node in possession of the same set independently computes the same winner (theorem 5.4). Second, version vectors provide a compact summary of each node’s history state, enabling the gossip protocol to compute precisely which mutations a peer is missing in $O(N)$ time (section 6). Third, Ed25519 signatures bound to canonical JSON encodings ensure that only authorised nodes can produce valid mutations and only moderator-credentialed clients can access the store (section 4).

The 4-step symmetric gossip protocol (section 7) is deadlock-free, idempotent, and bandwidth-efficient: in steady state it transfers only the ΔM mutations since the last sync with a given peer. The expected cluster-wide convergence time is $\Theta(N \log N)$ rounds for fanout-1 gossip, converging to a few minutes for practical small-to-medium clusters (table 6).

The eight core invariants (section 13) are enforced structurally by the implementation: global uniqueness by sequence collision detection, immutability by TypeScript readonly types, authenticity by mandatory signature verification on all network paths, and monotonicity by the max-based vector advancement rule.

PEBBLE is intentionally minimal in v0.1: no leader election, no consensus, no dynamic membership, no transport encryption. These choices make the system easy to reason about, easy to deploy, and easy to extend. The limitations documented in section 14 and the roadmap in section 15 chart a clear path toward a more capable system that preserves the leaderless, eventually consistent design philosophy.

Implementation status. PEBBLE is implemented in TypeScript on Node.js, structured as a pnpm monorepo of seven packages. The test suite comprises 121 automated tests covering unit logic, failure modes, and end-to-end multi-node scenarios including 3-node convergence, network partition healing, crash recovery, security rejection of forged mutations, and randomised chaos testing across hundreds of concurrent writes.

Availability. The source code is available at <https://github.com/neon-x-hub/pebble> under the MIT licence.

References

- [1] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang. High-speed high-security signatures. *Journal of Cryptographic Engineering*, 2(2):77–89, 2012.
- [2] E. A. Brewer. Towards robust distributed systems. In *Proceedings of the 19th Annual ACM Symposium on Principles of Distributed Computing (PODC)*, 2000. Invited keynote.
- [3] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Voshall, and W. Vogels. Dynamo: Amazon’s highly available key-value store. In *Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP)*, pages 205–220, 2007.
- [4] A. Demers, D. Greene, C. Hauser, W. Irish, J. Larson, S. Shenker, H. Sturgis, D. Swinehart, and D. Terry. Epidemic algorithms for replicated database maintenance. In *Proceedings of the 6th Annual ACM Symposium on Principles of Distributed Computing (PODC)*, pages 1–12, 1987.
- [5] C. J. Fidge. Timestamps in message-passing systems that preserve the partial ordering. *Australian Computer Science Communications*, 10(1):56–66, 1988.
- [6] S. Gilbert and N. A. Lynch. Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2):51–59, 2002.
- [7] S. Josefsson and I. Liusvaara. Edwards-Curve Digital Signature Algorithm (EdDSA). RFC 8032, Internet Engineering Task Force, 2017.
- [8] A.-M. Kermarrec and M. van Steen. Gossiping in distributed systems. *ACM SIGOPS Operating Systems Review*, 41(5):2–7, 2007.
- [9] R. Klophaus. Riak core: Building distributed applications without shared state. In *ACM SIGPLAN Commercial Users of Functional Programming (CUFP)*, 2010.
- [10] S. Kulkarni, M. Demirbas, D. Madappa, B. Avva, and M. Leone. Logical physical clocks and consistent snapshots in globally distributed databases. In *Proceedings of the 18th International Conference on Principles of Distributed Systems (OPODIS)*, 2014.
- [11] L. Lamport. Paxos made simple. *ACM SIGACT News*, 32(4):18–25, 2001.
- [12] F. Mattern. Virtual time and global states of distributed systems. *Parallel and Distributed Algorithms*, pages 215–226, 1989.
- [13] D. Ongaro and J. Ousterhout. In search of an understandable consensus algorithm. In *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC)*, pages 305–319, 2014.
- [14] A. Rundgren, B. Jordan, and S. Erdtman. JSON Canonicalization Scheme (JCS). RFC 8785, Internet Engineering Task Force, 2020.